In this section you'll learn how to unit-test API controllers and minimal API endpoints. You'll learn about the benefits and difficulties of testing these components in isolation and the situations when it can be useful.

Unit tests are all about isolating behavior; you want to test only the logic contained in the component itself, separate from the behavior of any dependencies. The Razor Pages and MVC/API frameworks use the filter pipeline, routing, and model-binding systems, but these are all external to the controller or `PageModel`s. The `PageModel`s and controllers themselves are responsible for a limited number of things:

- For invalid requests (that have failed validation, for example), return an appropriate `ActionResult` (API controllers) or redisplay a form (Razor Pages).
- For valid requests, call the required business logic services and return an appropriate `ActionResult` (API controllers), or show or redirect to a success page (Razor Pages).
- Optionally, apply resource-based authorization as required.

Controllers and Razor Pages generally shouldn't contain business logic themselves; instead, they should call out to other services. Think of them more as orchestrators, serving as the intermediary between the HTTP interfaces your app exposes and your business logic services.

If you follow this separation, you'll find it easier to write unit tests for your business logic, and you'll benefit from greater flexibility when you want to change your controllers to meet your needs. With that in mind, there's often a drive to make your controllers and page handlers as thin as possible, to the point where there's not much left to test!